

- [47] Subham Sekhar Sahoo, Anselm Paulus, Marin Vlastelica, Vít Musil, Volodymyr Kuleshov, and Georg Martius. Backpropagation through Combinatorial Algorithms: Identity with Projection Works. In *The Eleventh International Conference on Learning Representations*, February 2023.
- [48] Michael Schlichtkrull, Thomas N Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In Aldo Gangemi, Roberto Navigli, Maria-Esther Vidal, Pascal Hitzler, Raphaël Troncy, Laura Hollink, Anna Tordai, and Mehwish Alam, editors, *The Semantic Web*, pages 593–607, Cham, 2018. Springer International Publishing. ISBN 978-3-319-93417-4.
- [49] John Schulman, Nicolas Heess, Theophane Weber, and Pieter Abbeel. Gradient estimation using stochastic computation graphs. In *Advances in Neural Information Processing Systems*, 2015.
- [50] Dimitar Shterionov, Joris Renkens, Jonas Vlasselaer, Angelika Kimmig, Wannes Meert, and Gerda Janssens. The Most Probable Explanation for Probabilistic Logic Programs with Annotated Disjunctions. In Jesse Davis and Jan Ramon, editors, *Inductive Logic Programming*, Lecture Notes in Computer Science, pages 139–153, Cham, 2015. Springer International Publishing. ISBN 978-3-319-23708-4. doi: 10.1007/978-3-319-23708-4_10.
- [51] Jakub M. Tomczak. *Deep Generative Modeling*. Springer International Publishing, Cham, 2022. ISBN 978-3-030-93157-5 978-3-030-93158-2. doi: 10.1007/978-3-030-93158-2.
- [52] Frank van Harmelen and Annette ten Teije. A Boxology of Design Patterns for Hybrid Learning and Reasoning Systems. *Journal of Web Engineering*, 18(1):97–124, 2019. ISSN 1540-9589. doi: 10.13052/jwe1540-9589.18133.
- [53] Emile van Krieken, Jakub Tomczak, and Annette Ten Teije. Stochastic: A framework for general stochastic automatic differentiation. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 7574–7587. Curran Associates, Inc., 2021.
- [54] Emile van Krieken, Erman Acar, and Frank van Harmelen. Analyzing differentiable fuzzy logic operators. *Artificial Intelligence*, 302:103602, 2022. ISSN 0004-3702. doi: 10.1016/j.artint.2021.103602.
- [55] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need, December 2017.
- [56] Thomas Winters, Giuseppe Marra, Robin Manhaeve, and Luc De Raedt. DeepStochLog: Neural Stochastic Logic Programming. In *Proceedings of the First MiniCon Conference*, February 2022.
- [57] Jingyi Xu, Zilu Zhang, Tal Friedman, Yitao Liang, and Guy den Broeck. A semantic loss function for deep learning with symbolic knowledge. In Jennifer Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80, pages 5502–5511, Stockholmsmässan, Stockholm Sweden, 2018. PMLR.
- [58] Zhun Yang, Adam Ishay, and Joohyung Lee. NeurASP: Embracing neural networks into answer set programming. In Christian Bessiere, editor, *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI-20*, pages 1755–1762. International Joint Conferences on Artificial Intelligence Organization, July 2020. doi: 10.24963/ijcai.2020/243.
- [59] Chih-Kuan Yeh, Been Kim, Serkan O. Arik, Chun-Liang Li, Tomas Pfister, and Pradeep Ravikumar. On Completeness-aware Concept-Based Explanations in Deep Neural Networks, February 2022.
- [60] Choi Yoojung, Antonio Vergari, and Guy Van den Broeck. Probabilistic circuits: A unifying framework for tractable probabilistic models. 2020.

A Derivation of prediction-only variant loss

$$\mathbb{E}_{p(\mathbf{P})} [-\mathbb{E}_{p(\mathbf{y}|\mathbf{P})} [\log q_\phi(\mathbf{y}|\mathbf{P})]] \quad (10)$$

$$= -\mathbb{E}_{p(\mathbf{P})} [\mathbb{E}_{p(\mathbf{w}, \mathbf{y}|\mathbf{P})} [\log q_\phi(\mathbf{y}|\mathbf{P})]] \quad (11)$$

$$= -\mathbb{E}_{p(\mathbf{P}, \mathbf{w})} [\log q_\phi(c(\mathbf{w})|\mathbf{P})] \quad (12)$$

$$\mathcal{L}_{Pred}(\phi) = -\log q_\phi(c(\mathbf{w})|\mathbf{P}), \quad \mathbf{P}, \mathbf{w} \sim p(\mathbf{P}, \mathbf{w}) \quad (13)$$

In line 11, we marginalize out $\mathbf{w}$, and use the fact that $\mathbf{y}$ is deterministic given $\mathbf{w}$.

B Constrained structured output prediction

We consider how to implement the constrained structured output prediction task considered in (for example) [2, 1, 57] in the A-NESI framework. Here, the goal is to learn a mapping of some X to a structured *output* space W , where we have some constraint $c(\mathbf{w})$ that returns 1 if the background knowledge holds, and 0 otherwise. We can model the constraints using $Y = \{0, 1\}$; that is, the ‘output’ in our problem setup is whether $\mathbf{w}$ satisfies the background knowledge c or not. We give an example of this setting in Figure 6.

Then, we design the inference model as follows. 1) $q_\phi(y|\mathbf{P})$ is tasked with predicting the probability that randomly sampled outputs $\mathbf{w} \sim p(\mathbf{w}|\mathbf{P})$ will satisfy the background knowledge. 2) $q_\psi(\mathbf{w}|y=1, \mathbf{P})$ is an approximate posterior over structured outputs $\mathbf{w}$ that satisfy the background knowledge c .

This setting changes the interpretation of the set W from *unobserved* worlds to *observed* outputs. We will train our perception module using a “strongly” supervised learning loss where $\mathbf{x}, \mathbf{w} \sim \mathcal{D}_L$:

$$\mathcal{L}_{Perc}(\theta) = -\log q_\psi(\mathbf{w}|y=1, \mathbf{P} = f_\theta(\mathbf{x})). \quad (14)$$

If we also have unlabeled data $\mathcal{D}_U$, we can use the prediction model to ensure the perception model gives high probabilities for worlds that satisfy the background knowledge. This approximates Semantic Loss [57]: Given $\mathbf{x} \sim \mathcal{D}_U$,

$$\mathcal{L}_{SL}(\theta) = -\log q_\phi(y=1|\mathbf{P} = f_\theta(\mathbf{x})). \quad (15)$$

That is, we have some input $\mathbf{x}$ for which we have no labelled output. Then, we increase the probability that the belief $\mathbf{P}$ the perception module f_θ predicts for $\mathbf{x}$ would sample structured outputs $\mathbf{w}$ that satisfy the background knowledge.

Training the inference model in this setting can be challenging if the problem is very constrained. Then, random samples $\mathbf{P}, \mathbf{w} \sim p(\mathbf{P}, \mathbf{w})$ will usually not satisfy the background knowledge. Since we are only in the case that $y=1$, we can choose to sample from the inference model q_ϕ and exploit the symbolic pruner to obtain samples that are guaranteed to satisfy the background knowledge. Therefore, we modify equation 8 to the *on-policy joint matching loss*

$$\mathcal{L}_{Expl}(\mathbf{P}, \phi, \psi) = \mathbb{E}_{q_\psi(\mathbf{w}|y=1, \mathbf{P})} \left[\left(\log \frac{q_{\phi, \psi}(\mathbf{w}, y=1|\mathbf{P})}{p(\mathbf{w}|\mathbf{P})} \right)^2 \right] \quad (16)$$

Here, we incur some sampling bias by not sampling structured outputs from the true posterior, but this bias will reduce as q_ϕ becomes more accurate. We can also choose to combine the on- and off-policy losses. Another option to make learning easier is using the suggestions of Section 3.2.2: factorize y to make it more fine-grained.

C A-NESI as a Gradient Estimation method

In this appendix, we discuss using the method A-NESI introduced for general gradient estimation [40]. We first define the gradient estimation problem. Consider some neural network f_θ that predicts the parameters $\mathbf{P}$ of a distribution over unobserved variable $\mathbf{z} \in Z$: $p(\mathbf{z}|\mathbf{P} = f_\theta(\mathbf{x}))$. This distribution corresponds to the distribution over worlds $p(\mathbf{w}|\mathbf{P})$ in A-NESI. Additionally, assume we have some deterministic function $g(\mathbf{z})$ that we want to maximize in expectation. This maximization requires estimating the gradient

$$\nabla_\theta \mathbb{E}_{p(\mathbf{z}|\mathbf{P}=f_\theta(\mathbf{x}))} [g(\mathbf{z})]. \quad (17)$$

Common methods for estimating this gradient are reparameterization [26], which only applies to continuous random variables and differentiable r , and the score function [40, 49] which has notoriously high variance.

Instead, our gradient estimation method learns an *inference model* $q_\phi(r|\mathbf{P})$ to approximate the distribution of outcomes $r = g(\mathbf{z})$ for a given $\mathbf{P}$. In A-NESI, this is the prediction network $q_\phi(\mathbf{y}|\mathbf{P})$ that estimates the WMC problem of Equation 1. Approximating a *distribution* over outcomes is similar to the idea of distributional reinforcement learning [8]. Our approach is general: Unlike reparameterization, we can use inference models in settings with discrete random variables $\mathbf{z}$ and non-differentiable downstream functions g .

We derive the training loss for our inference model similar to that in Section 3.2.1. First, we define the joint on latents $\mathbf{z}$ and outcomes r like the joint process in 6 as $p(r, \mathbf{z}|\mathbf{P}) = p(\mathbf{z}|\mathbf{P}) \cdot \delta_{g(\mathbf{z})}(r)$, where $\delta_{g(\mathbf{z})}(r)$ is the dirac-delta distribution that checks if the output of g on $\mathbf{z}$ is equal to r . Then we introduce a prior over distribution parameters $p(\mathbf{P})$, much like the prior over beliefs in A-NESI. An obvious choice is to use a prior conjugate to $p(\mathbf{z}|\mathbf{P})$. We minimize the expected KL-divergence between $p(r|\mathbf{P})$ and $q_\phi(r|\mathbf{P})$:

$$\mathbb{E}_{p(\mathbf{P})}[D_{KL}(p||q_\phi)] \quad (18)$$

$$= \mathbb{E}_{p(\mathbf{P})} [\mathbb{E}_{p(r|\mathbf{P})} [\log q_\phi(r|\mathbf{P})]] + C \quad (19)$$

$$= \mathbb{E}_{p(\mathbf{P})} \left[\int_{\mathbb{R}} p(r|\mathbf{P}) \log q_\phi(r|\mathbf{P}) dr \right] + C \quad (20)$$

Next, we marginalize over $\mathbf{z}$, dropping the constant:

$$\mathbb{E}_{p(\mathbf{P})} \left[\int_{\mathbf{Z}} \int_{\mathbb{R}} p(r, \mathbf{z}|\mathbf{P}) \log q_\phi(r|\mathbf{P}) dr d\mathbf{z} \right] \quad (21)$$

$$= \mathbb{E}_{p(\mathbf{P})} \left[\int_{\mathbf{Z}} p(\mathbf{z}|\mathbf{P}) \int_{\mathbb{R}} \delta_{g(\mathbf{z})}(r) \log q_\phi(r|\mathbf{P}) dr d\mathbf{z} \right] \quad (22)$$

$$= \mathbb{E}_{p(\mathbf{P})} \left[\int_{\mathbf{Z}} p(\mathbf{z}|\mathbf{P}) \log q_\phi(g(\mathbf{z})|\mathbf{P}) d\mathbf{z} \right] \quad (23)$$

$$= \mathbb{E}_{p(\mathbf{P}, \mathbf{z})} [\log q_\phi(g(\mathbf{z})|\mathbf{P})] \quad (24)$$

This gives us a negative-log-likelihood loss function similar to Equation 7.

$$\mathcal{L}_{Inf}(\phi) = -\log q_\phi(g(\mathbf{z})|\mathbf{P}), \quad \mathbf{P}, \mathbf{z} \sim p(\mathbf{z}, \mathbf{P}) \quad (25)$$

where we sample from the joint $p(\mathbf{z}, \mathbf{P}) = p(\mathbf{P})p(\mathbf{z}|\mathbf{P})$.

We use a trained inference model to get gradient estimates:

$$\nabla_{\mathbf{P}} \mathbb{E}_{p(\mathbf{z}|\mathbf{P})}[g(\mathbf{z})] \approx \nabla_{\mathbf{P}} \mathbb{E}_{q_\phi(r|\mathbf{P})}[r] \quad (26)$$

We use the chain rule to update the parameters θ . This requires a choice of distribution $q_\phi(r|\mathbf{P})$ for which computing the mean $\mathbb{E}_{q_\phi(r|\mathbf{P})}[r]$ is easy. The simplest option is to parameterize q_ϕ with a univariate normal distribution. We predict the mean and variance using a neural network with parameters ϕ . For example, a neural network m_ϕ would compute $\mu = m_\phi(\mathbf{P})$. Then, the mean parameter is the expectation on the right-hand side of Equation 26. The loss function for f_θ with this parameterization is:

$$\mathcal{L}_{NN}(\theta) = -m_\phi(f_\theta(\mathbf{x})), \quad \mathbf{x} \sim \mathcal{D} \quad (27)$$

Interestingly, like A-NESI, this gives zero-variance gradient estimates! Of course, bias comes from the error in the approximation of q_ϕ .

Like A-NESI, we expect the success of this method to rely on the ease of finding a suitable prior over $\mathbf{P}$ to allow proper training of the inference model. See the discussion in Section 3.2.3. We also expect that, like in A-NESI, it will be easier to train the inference model if the output $r = g(\mathbf{z})$ is structured instead of a single scalar. We refer to Section 3.2.2 for this idea. Other challenges might be stochastic and noisy output measurements of r and non-stationarity of g , for instance, when training a VAE [26].